IndustryOR 問題 1：旅行参加児童選択問題の AMPL による数理モデリングと最適化

lecture2026 演習資料

2026 年 10 月 13 日

概要

本稿では、IndustryOR ベンチマークに含まれる「張家の旅行参加児童選択問題（Problem 1: Children Vacation Selection Problem）」を対象とし、0-1 整数計画問題（Binary Integer Programming: BIP）としての定式化、数理モデリング言語 AMPL による実装、および Gurobi ソルバーを用いた最適化結果を詳細に解説する。特に、条件付き選択（論理含意・排他関係）を線形不等式に変換するモデリング技法に焦点を当て、得られた最適解（最小費用 $3,050）の妥当性を考察する。

1　問題の概要

1.1　問題の背景と設定

張（Zhang）家には 6 人の子ども（Harry, Hermione, Ron, Fred, George, Ginny）がおり、今度の旅行にどの子どもを連れて行くかを計画している。各子どもを連れて行くには個別の費用がかかる。夫妻は総費用を最小化するような同伴者の組み合わせを求めている。

各子どもの同伴費用は以下の表 1 の通りである。

表 1　各子どもの同伴費用

子ども (i)	記号表記	費用 c_i [$]
Harry	Ha	1,200
Hermione	He	1,650
Ron	Ro	750
Fred	Fr	800
George	Ge	800
Ginny	Gi	1,500

1.2　考慮すべき制約条件

夫妻は以下の諸条件を満たす必要がある：

1. **最大人数制約**: 同伴できる子どもの人数は最大 4 人までである。

2. **最小人数制約**: 費用はかかるものの、夫妻は少なくとも 3 人の子どもを連れて行くことを決定している。
3. **必須参加条件**: Ginny は最年少であるため、張家は必ず彼女を連れて行く。
4. **不仲条件 1**: Harry を連れて行く場合、Harry は Fred と仲が悪いため、Fred は連れて行かない。
5. **不仲条件 2**: Harry を連れて行く場合、Harry は George とも仲が悪いため、George は連れて行かない。
6. **連動条件 1**: George を連れて行く場合、必ず Fred も連れて行かなければならない。
7. **連動条件 2**: George を連れて行く場合、必ず Hermione も連れて行かなければならない。

2 数理最適化モデルの定式化

本問題は、各子どもを「連れて行く（1）」か「連れて行かない（0）」かを決定する **0-1 整数計画問題（Binary IP）**として定式化できる。

2.1 集合とパラメータ

- 集合 $C = \{\text{Harry}, \text{Hermione}, \text{Ron}, \text{Fred}, \text{George}, \text{Ginny}\}$：子どもの集合
- パラメータ $c_i \geq 0$ $(i \in C)$：子ども i を連れて行く費用

2.2 決定変数

各子ども $i \in C$ について、以下の二値決定変数（0-1 変数）を定義する：

$$x_i \in \{0, 1\} \quad (\forall i \in C) \tag{1}$$

ここで、$x_i = 1$ は子ども i を旅行に同伴することを表し、$x_i = 0$ は同伴しないことを表す。

2.3 目的関数

旅行に連れて行く子どもの総費用を最小化する：

$$\min \quad Z = \sum_{i \in C} c_i x_i \tag{2}$$

具体的には以下の線形和となる：

$$Z = 1200x_{\text{Ha}} + 1650x_{\text{He}} + 750x_{\text{Ro}} + 800x_{\text{Fr}} + 800x_{\text{Ge}} + 1500x_{\text{Gi}}$$

2.4 制約条件とその線形不等式表現

論理条件を MIP（混合整数計画）で表現する標準的な手法に基づき、制約を定式化する。

2.4.1　人数制約（最大・最小）

同伴人数は 3 人以上かつ 4 人以下である：

$$\sum_{i \in C} x_i \leq 4 \quad \text{（最大 4 人）} \tag{3}$$

$$\sum_{i \in C} x_i \geq 3 \quad \text{（最小 3 人）} \tag{4}$$

2.4.2　必須参加条件

Ginny は必ず同伴する：

$$x_{\mathrm{Gi}} = 1 \tag{5}$$

2.4.3　排他・不仲条件（At Most One）

「Harry を連れて行くならば Fred は連れて行かない（$x_{\mathrm{Ha}} = 1 \implies x_{\mathrm{Fr}} = 0$）」、および「Harry を連れて行くならば George は連れて行かない（$x_{\mathrm{Ha}} = 1 \implies x_{\mathrm{Ge}} = 0$）」は、2 人のうち同時に選べるのは高々 1 人であることを意味するため、次の不等式となる：

$$x_{\mathrm{Ha}} + x_{\mathrm{Fr}} \leq 1 \tag{6}$$

$$x_{\mathrm{Ha}} + x_{\mathrm{Ge}} \leq 1 \tag{7}$$

2.4.4　論理含意条件（Implication: $A \implies B$）

「George を連れて行くならば Fred も連れて行く（$x_{\mathrm{Ge}} = 1 \implies x_{\mathrm{Fr}} = 1$）」は、$x_{\mathrm{Ge}} \leq x_{\mathrm{Fr}}$ と表現される。同様に「George を連れて行くならば Hermione も連れて行く（$x_{\mathrm{Ge}} = 1 \implies x_{\mathrm{He}} = 1$）」は $x_{\mathrm{Ge}} \leq x_{\mathrm{He}}$ と表される：

$$x_{\mathrm{Ge}} - x_{\mathrm{Fr}} \leq 0 \tag{8}$$

$$x_{\mathrm{Ge}} - x_{\mathrm{He}} \leq 0 \tag{9}$$

3　AMPL による実装

AMPL（A Modeling Language for Mathematical Programming）では、モデルの論理構造（`.mod`）、数値データ（`.dat`）、および解法コマンド（`.run`）を綺麗に分離して記述できる。

3.1　モデルファイル (`problem1.mod`)

```
# Problem 1: Children Vacation Selection (IP)
set CHILDREN;

param cost{CHILDREN} >= 0;

var take{CHILDREN} binary;
```

```
minimize TotalCost:
    sum {c in CHILDREN} cost[c] * take[c];

subject to MaxChildren:
    sum {c in CHILDREN} take[c] <= 4;

subject to MinChildren:
    sum {c in CHILDREN} take[c] >= 3;

subject to MustTakeGinny:
    take["Ginny"] = 1;

subject to ConflictHarryFred:
    take["Harry"] + take["Fred"] <= 1;

subject to ConflictHarryGeorge:
    take["Harry"] + take["George"] <= 1;

subject to GeorgeRequiresFred:
    take["George"] <= take["Fred"];

subject to GeorgeRequiresHermione:
    take["George"] <= take["Hermione"];
```

3.2 データファイル (problem1.dat)

```
set CHILDREN := Harry Hermione Ron Fred George Ginny;

param cost :=
    Harry     1200
    Hermione  1650
    Ron        750
    Fred       800
    George     800
    Ginny     1500;
```

3.3 実行スクリプト (problem1.run)

```
reset;
model problem1.mod;
data problem1.dat;

option solver '/Users/mikiokubo/Documents/ampl/ampl.macos64/gurobi';
solve;

display TotalCost;
display take;
```

3.4 Python API (amplpy) による統合実装

Python から直接モデル定義・データ設定・求解・結果集計を一元管理する場合、amplpy を用いて以下のように記述できる。

```
from amplpy import AMPL, modules

modules.load()
ampl = AMPL()
ampl.set_option("solver", "highs")

ampl.eval("""
set CHILDREN;
param cost{CHILDREN} >= 0;
var take{CHILDREN} binary;

minimize TotalCost:
    sum {c in CHILDREN} cost[c] * take[c];

subject to MinChildren:
    sum {c in CHILDREN} take[c] >= 3;
subject to MaxChildren:
    sum {c in CHILDREN} take[c] <= 4;
subject to MustTakeGinny:
    take["Ginny"] = 1;
subject to ConflictHarryFred:
    take["Harry"] + take["Fred"] <= 1;
subject to ConflictHarryGeorge:
    take["Harry"] + take["George"] <= 1;
subject to GeorgeRequiresFred:
    take["George"] <= take["Fred"];
subject to GeorgeRequiresHermione:
    take["George"] <= take["Hermione"];
""")

children = ["Harry", "Hermione", "Ron", "Fred", "George", "Ginny"]
costs = {"Harry": 1200, "Hermione": 1650, "Ron": 750, "Fred": 800, "George":
    800, "Ginny": 1500}

ampl.set["CHILDREN"] = children
ampl.param["cost"] = costs
ampl.solve()
```

4　最適化実行と結果

4.1　ソルバー実行ログ

AMPL 経由で Gurobi 12.0.1 を呼び出して求解した結果は以下の通りである。

```
Gurobi 12.0.1: optimal solution; objective 3050
0 simplex iterations

========================================
Optimal Objective Value: 3050
========================================
Harry       : 0 (Cost: 1200)
Hermione    : 0 (Cost: 1650)
Ron         : 1 (Cost: 750)
Fred        : 1 (Cost: 800)
George      : 0 (Cost: 800)
Ginny       : 1 (Cost: 1500)
========================================
```

4.2　最適解の構成

最適化の結果、選択された子どもおよび費用は以下の通りである：

- **Ginny**: 選択（$1,500）— 必須参加
- **Ron**: 選択（$750）— 費用最小候補
- **Fred**: 選択（$800）— 次点費用候補
- **Harry, Hermione, George**: 不参加（各 $0）

最小総費用: **$3,050**

5　制約充足の検証と考察

得られた最適解がすべての制約条件を満たしているか確認する。

1. **人数**: 同伴者数は Ginny, Ron, Fred の計 3 人であり、$3 \leq 3 \leq 4$ を満たす。
2. **Ginny**: $x_{\mathrm{Gi}} = 1$ であり、条件を満たす。
3. **Harry と Fred/George の排他**: $x_{\mathrm{Ha}} = 0$ であるため、Fred や George との衝突は発生しない。
4. **George の連動条件**: $x_{\mathrm{Ge}} = 0$ であるため、Fred や Hermione を強制的に追加する必要はない。

5.1 George を選ばなかった理由の数理的背景

George の費用（$800）は Fred（$800）と同じであり、Ron（$750）に次ぐ低廉な水準にある。しかし、もし George を同伴させた場合（$x_{\mathrm{Ge}} = 1$）：

- 連動条件により Fred（$800）と Hermione（$1,650）が強制的に選択される（$x_{\mathrm{Fr}} = 1, x_{\mathrm{He}} = 1$）。
- 必須の Ginny（$1,500）と合わせると、この時点ですでに 4 人となり、費用は $1500 + 800 + 800 + 1650 = \$4,750$ に達する。

したがって、連動制約が課されている George を選ばず、単独で選べる Fred と最安の Ron を選ぶことで、最低人数の 3 人にとどめ、費用を $3,050 に抑える解が厳密に最適となる。

6 まとめ

本問題は、論理的な前提条件（If-Then）や排他選択（At-Most-One）を含む典型的な意思決定問題であり、0-1 整数計画問題として自然に表現できる。AMPL を用いることで簡潔にモデルを構築し、最新の数理最適化ソルバー（Gurobi）によって瞬時に大域的最適解（$3,050）を導出できることが確認された。